

MODULE 06 · DOMAIN 3 - DEBUGGING AND PERFORMANCE

Making It Faster

The query is correct. It takes eleven minutes. Both of those must still be true afterwards.

DOMAIN 3 of 5 MODULE 6 of 11 PROMPTS 3

A query that got four times faster and slightly **different**.

Why this goes wrong more than debugging does

- ◆ **Success is measurable.** The clock went down. That feels like proof.
- ◆ **Correctness is not checked.** Nobody reruns the old query to compare.
- ◆ **Rewrites change semantics quietly.** A join becomes a subquery. Nulls behave differently.
- ◆ **The old output disappears.** After one run you have nothing to compare against.

DO THIS FIRST

Save the current output before you change anything. That is the entire safety net.

Read the plan. Ranked by cost, not described.

What actually matters in a plan

- ◆ **Estimated rows against actual.** A large gap means the optimiser is working blind.
- ◆ **Data movement.** Broadcasts and redistributions on big tables dominate everything else.
- ◆ **Nested loops on large inputs.** Almost always a join condition the planner could not use.
- ◆ **The same table scanned repeatedly.** Cheap per scan, expensive in total.

THE POINT

Everything else in a plan is detail. These four account for most of the time in most slow queries.

What you actually type

Ranked, quantified, and no rewrites yet.

Below is the query and its execution plan. Do not rewrite anything yet.

Rank the plan steps by estimated share of total cost, highest first. Give me at most five. For each one state:

- Which line of the query it corresponds to
- Why it is expensive, in terms of the plan not the SQL
- Whether the row estimate looks wrong, and by how much

Then list separately any step whose cost comes from a stale or missing statistic rather than from the query.

If the plan does not support a claim, do not make it.

THE LINE THAT SHAPES IT

Capping it at five forces a ranking, not a walkthrough.

Four findings, one procedure

A consumption load with three time windows.

- ```
01 1. Temp table scanned four times
02 Three window CTEs plus a DISTINCT driver. Same rows each time.
03
04 2. Hash function called inside a join predicate
05 Computed per row. Planner cannot use it as a hash join key.
06
07 3. Date conversion applied per row, then filtered
08 TO_DATE on every row before the window predicate runs.
09
10 4. Unbounded aggregation over full history
11 No date predicate. Scans everything, every night.
```

## WHAT KIND OF FIX EACH IS

Only the first two are rewrites. The third is a predicate change and the fourth is a question for the business.

Rewrite it. But say what must not change.

# Pin these down before you ask

- ◆ **The grain.** One row per what. Same sentence from module three.
- ◆ **The row count.** State the number. It is the first thing you will check.
- ◆ **Null behaviour.** Which columns can be null, and what a null must produce.
- ◆ **Ordering guarantees.** If anything downstream depends on order, say so.

## THE POINT

An unconstrained rewrite is not wrong to ask for. It is just impossible to check.

# What you actually type

The constraints come before the request.

Rewrite the query below for performance. These must not change:

- Grain: one row per part per site
- Row count: 412,806
- Null handling: a part with no movements returns 0, not null

Target the four findings from the plan analysis above.

For each change you make, state in one line what it changes about execution and why the result set is unaffected.

If any change could alter the result, do not make it. List it separately as something I would have to decide about.

## WHY THE LAST BLOCK EXISTS

The last instruction is the important one. It gives it somewhere to put the risky ideas.

# Four scans become one

Three nested windows over the same rows.

```
01 -- Before: three CTEs, each scanning #wmov
02 cte_12: WHERE postdt BETWEEN <12 months> ... GROUP BY part,site
03 cte_24: WHERE postdt BETWEEN <24 months> ... GROUP BY part,site
04 cte_60: WHERE postdt BETWEEN <60 months> ... GROUP BY part,site
05
06 -- After: one scan, conditional aggregation
07 SELECT part, site,
08 SUM(CASE WHEN postdt >= <12 months> THEN net ELSE 0 END) AS m12,
09 SUM(CASE WHEN postdt >= <24 months> THEN net ELSE 0 END) AS m24,
10 SUM(net) AS m60
11 FROM #wmov WHERE postdt >= <60 months> GROUP BY part, site
```

## WHY IT IS SAFE

The windows are nested, so the widest one bounds the scan and the narrower ones are case expressions.

It argued that the results are the same.  
Now **prove** it.

# Why the argument is not enough

- ◆ The reasoning is usually correct. That is what makes this hard to insist on.
- ◆ It reasons about the query, not about your data.
- ◆ Edge cases live in the data. Nulls, duplicates, zero rows, boundary dates.
- ◆ One query settles it in the time it takes to read the argument.

## THE POINT

This is the same move as the debugging module. Ask for the check, not the claim.

# What you actually type

Both directions. That is the part people miss.

Below are the original query and your rewrite. Write me the queries that prove they return identical results.

1. Row count from each, side by side
2. Rows in original but not in rewrite
3. Rows in rewrite but not in original
4. Sum of each numeric measure from both, side by side

Queries two and three must both be included. One direction alone does not prove equivalence.

Then list the edge cases these four queries would NOT catch.

## THE HONEST LINE

The last line is the honest one. Four queries is a strong check, not a proof.

# Both directions, one result

If either count is non-zero, the rewrite is not equivalent.

```
01 SELECT 'in original only' AS side, COUNT(*) FROM (
02 SELECT * FROM original_result
03 EXCEPT
04 SELECT * FROM rewrite_result)
05 UNION ALL
06 SELECT 'in rewrite only', COUNT(*) FROM (
07 SELECT * FROM rewrite_result
08 EXCEPT
09 SELECT * FROM original_result)
10
11 -- Both zero, or the rewrite is not a rewrite.
```

## WHAT IT CAUGHT

It found 41 rows in the rewrite only. Parts with no movements now returned 0 instead of no row.

# Argued against proved

## WHAT YOU GET BY DEFAULT

### An argument

- Reasoning about the query
- Usually correct
- Free and instant
- Cannot see your data

## WHAT YOU SHOULD ASK FOR

### A test

- Reasoning about the result
- Correct or not, definitively
- Costs one query run
- Finds the forty one rows

## THE TIEBREAKER

The argument is worth reading. It is just not the thing you ship on.

The plan analysis ranks a broadcast join as the top cost. It also notes the row estimate on that step is 4,000 where the actual is 2.1 million.

You have a rewrite ready that removes the broadcast.

### What should you try first?

- ☐ A Apply the rewrite. The broadcast is the top cost
- ☐ B Update the table statistics and re-read the plan
- ☐ C Add a distribution key to the smaller table
- ☐ D Split the query into two steps with a temp table

### ◆ PAUSE AND DECIDE

## What should you try first?

- A

 Apply the rewrite  
*You would be optimising around a decision the planner made on bad information.*
- C

 Add a distribution key  
*Possibly right eventually. Still a change made while the planner has wrong numbers.*
- B

 Update stats, re-read  
*A 500x estimate error means the planner chose the broadcast blind. Fix the input first.*
- D

 Split into two steps  
*Materialising can help, and it is a workaround for a planner problem you have not diagnosed.*

### THE SIGNAL

A five hundred fold estimate error is the finding. The broadcast is the symptom of it.

The rewrite comes back with a clear justification for every change. It also lists one change separately, noting it could alter results for rows where a value is null.

That change accounts for most of the speed-up. Runtime goes from eleven minutes to ninety seconds.

### What do you do with the flagged change?

- ☐ A Take it. It flagged the risk, so the risk is known
- ☐ B Drop it and accept the smaller speed-up
- ☐ C Test whether any rows actually have that null
- ☐ D Take it and add a null guard to be safe

### ◆ PAUSE AND DECIDE

## What do you do with the flagged change?

- A

 Take it, risk is known  
*Knowing a risk exists is not knowing whether it applies. Those are different things.*
- B

 Drop it  
*Throws away most of the gain to avoid a risk you can measure in one count query.*
- C

 Test for the null  
*One count. If no rows have it, take the change. If some do, you have a real decision.*
- D

 Add a null guard  
*A guard changes behaviour too, and you still do not know what the old behaviour was.*

### THE SIGNAL

The prompt did its job by separating that change out. Your job is to settle it with a query.

The equivalence check returns zero rows in both directions. Counts match. Measure sums match to the penny.

A colleague asks whether the rewrite is proven correct.

### What is the accurate answer?

- ☐ A Yes. Both set differences are empty
- ☐ B Yes for today's data, which is not the same thing
- ☐ C No, because set difference ignores duplicate counts
- ☐ D No, until it has run in production for a week

### ◆ PAUSE AND DECIDE

## What is the accurate answer?

- A

 Yes, proven  
*Overstates it. You proved the two agree on one data set on one day.*
- C

 Duplicates are ignored  
*True of set difference, and the row counts matching already covers it here.*
- B

 Yes for today's data  
*The precise answer. Tomorrow may contain a boundary case today did not.*
- D

 Not until a week in production  
*Sets an unmeetable bar. By that logic nothing is ever verified before it ships.*

### THE SIGNAL

This is not pedantry. Date boundary logic is the classic case that passes today and fails next month.

The plan analysis says the largest single cost is a nightly full-table update on a history table, run to refresh dimension keys.

That update is not in the query you were asked to optimise. It is in the procedure that loads it.

### What is the right response?

- ☐ A Out of scope. Optimise the query you were given
- ☐ B Rewrite the update as part of the same change
- ☐ C Report it, with the cost figure, and ask how to proceed
- ☐ D Scope the update to only the current snapshot date

### ◆ PAUSE AND DECIDE

## What is the right response?

- A

 Stay in scope  
*You found the biggest cost in the workload and said nothing. Scope discipline taken too far.*
- B

 Rewrite it too  
*Expanding a performance ticket into a history table rewrite, unasked, is how changes go wrong.*
- C

 Report it with numbers  
*You have a measured finding outside your scope. Reporting it is the work; deciding is not yours.*
- D

 Scope it to today  
*Probably the right fix, and it is still a behaviour change to a history table made on your own.*

### THE SIGNAL

A finding outside your scope is still a finding. Handing it over with a number attached is enough.

# Take this to work tomorrow

- ◆ Before any optimisation, save the current output somewhere you can query it.
- ◆ Read the plan before you read the SQL. Ask for five steps, ranked.
- ◆ State grain, row count and null handling in the rewrite prompt itself.
- ◆ Run the equivalence check both directions. Every time, including when you are sure.

## THE PAYOFF

The first habit is the one that makes the last one possible.

Fast and correct. In that order of difficulty,  
and the **reverse** order of attention.